DEFENSE 1:

1. Introduction

1.1 Context

This report is part of the S3 project at EPITA (2026/2027), supervised by David Bouchet. It presents the work done by our group for the first defense of the **OCR Word Search Solver** project.

The objective is to design, in C, an application that takes the image of a word search puzzle as input, recognizes the letters of the grid and the words of the list through optical character recognition (OCR), solves the puzzle, and displays and saves the solved grid. The application must run on the school's Linux environment and compile without warnings.

1.2 Problem statement

A word search is a grid of letters in which words are hidden along straight lines (horizontal, vertical or diagonal, in either direction). The solving step is algorithmically simple. The difficulty lies upstream: the program only receives an image, so it must locate the grid and the word list, split the image into individual letters, and identify each letter reliably before any search can start.

The subject defines three levels of difficulty for the input images:

- **Level 1:** clean image, capital letters, bordered grid and cells of identical size, a single column of words;
- **Level 2:** level 1 plus defects (poor contrast, noise, blur, random rotation);
- **Level 3:** level 2 plus extra content on the image, word lists on several columns, and grids without borders.

Our design choice is a single recognition pipeline for all levels, so that difficulty only affects how much preparation the image needs, never the recognition model itself.

1.3 Processing chain

The application follows the steps given by the subject:

1. loading the image and removing colors (grayscale, then black and white);
2. preprocessing (manual and automatic deskew, noise removal, contrast enhancement);
3. locating the grid, the word list and the cells;
4. extracting the image of each letter;
5. recognizing each letter with a neural network;
6. rebuilding the grid and the word list as arrays of characters;
7. solving the grid, then displaying and saving the result.

1.4 Work distribution

The project is divided into three parts, each owned by one member:

Member	Part	Scope
Julien	User interface	Image loading and preview, manual rotation, saving, buttons launching recognition and solving, display of the results
Josias	Neural network	Design of the network, training on letter images, saving and loading of the learned parameters, recognition of letters
Lucas	Image recognition	Preprocessing, location of the grid and the word list, splitting into cells and letters

The parts depend on each other: the interface supplies the image, the image processing turns it into letter images, the neural network turns these into characters, and the interface displays the result. To allow parallel work, the modules are strictly separated. The interface contains no recognition code, and the neural network only receives letter images and returns characters.

1.5 State at the first defense

At the time of the first defense, the interface is functional: an image can be loaded from a path, previewed, rotated by a chosen angle, and saved. The buttons that launch recognition and solving are in place, ready to be connected to the other modules. The state of the image processing and of the neural network is detailed in the sections written by their owners.

2. Role and Mission of Each Member

The project is split into three parts, each owned by one member. The parts communicate through simple interfaces (SDL surfaces, letter images, arrays of characters).

Member	Part	Mission
Julien Allane Allion	User interface	Let the user control the whole pipeline and see its results
Josias Legesse	Neural network	Turn the image of one letter into a character
Lucas Bret	Image recognition	Turn the loaded image into clean, located letter images

user interface. Loading an image from a typed or pasted path, scaled preview, manual rotation, saving the image, the preprocessing controls, and the “RUN OCR” and “SOLVE GRID” buttons. At the final defense, it also displays the recognized grid and the solved grid with the words circled. The interface contains no recognition code. It hands an image to the pipeline and displays what comes back.

neural network. Goal of the multi-layer neural network is to build the training set of letter images, train the network in a separate program, save and load the learned parameters, and write the function that returns the most probable character for a letter image. The network receives only letter images. It does not depend on where they come from or on the difficulty level.

image recognition. He converts the image to grayscale and black and white, applies the preprocessing (automatic deskew, noise removal, contrast), locates the grid, the word list and the cells, and extracts one image per letter. He also keeps each letter’s position, so the character grid and the word strings can be rebuilt after recognition.

Shared tasks. The Git repository, the Makefile (`all` and `clean`), the `README` and `AUTHORS` files, the test and training images, the integration of the modules behind “RUN OCR”, and the command-line solver (josias).

3. Objectives, Planning and Progress

3.1 Objectives

The final goal is an application that reads the image of a word search, recognizes the grid and the word list, solves the puzzle, and displays and saves the result. It must handle the three difficulty levels of the subject with one recognition pipeline.

#	Objective	Lead
O1	Interface: load, preview, correct and save an image, launch recognition and solving	Julien
O2	Preprocessing: grayscale, black and white, deskew, noise removal, contrast	Lucas
O3	Detection of the grid, the word list and the cells	Lucas
O4	Extraction of one image per letter	Lucas
O5	Neural network: design, training, saving and loading, recognition	Josias
O6	Reconstruction of the grid and the word list as arrays of characters	Lucas, Josias
O7	Command-line solver	Josias
O8	Display and saving of the solved grid with the words circled	Julien

Flexible assignment. The “Lead” column shows who is responsible for an objective and answers for its progress at the defense. It does not lock the work to one person.

O1 and O5 can advance in parallel, since the interface needs only an image and the network needs only letter images. O2 to O4 are the critical path: the network cannot work on a real image until the letters have been extracted.

3.2 Task planning

Phase	Period	Tasks	Defense
1. Setup	[17 - 21 october]	Repository, Makefile, module boundaries, test images	First
2. First modules	[21 - 30]	Interface (O1), basic preprocessing (O2), network prototype (O5)	First

3. Detection and recognition	not decided yet	Detection (O3), letter extraction (O4), network training, solver (O7)	Final
4. Integration	not decided yet	Connection to the buttons, solved grid display (O8), tests on the three levels	Final

3.3 Expected and actual progress

Objective	Expected for defense 1	Actual	Status
O1 Loading and preview	Load and display an image	Typed path, scaled preview	Done
O1 Manual rotation	Rotation by a given angle	Rotation about the center, always from the original	Done
O1 Saving	Save the displayed image	PNG export	Done
O1 Preprocessing options	Controls for deskew, noise, contrast	Controls present, not yet connected	Partial
O1 Drag and drop	Load by dropping a file	Removed (unreliable),	Dropped
O2 Preprocessing	Preprocess the image	Load image but no backend display	In progress
O3 Detection	Proper detection of the grid	Rotation backend	In progress
O4 Neural network	Learns the xnor function	Works as intended	Done
O5 Solver	Takes a matrix of letter and searches for the argument	Solves word search	Done

3.4 Analysis of the gaps

On the interface, the features required for this defense are in place. Drag and drop was abandoned because its event depends on the window manager and does not reach the

program on every system (virtual machines, Wayland). A feature that works on only some machines is a risk for a demonstration on the school computers. The preprocessing controls are displayed but only manual rotation is active, so the interface will not need to change when the processing modules are delivered. For the neural network, the features required for this defense are also in place, but some modifications could have been made regarding the presence of random values in loops that could become a problem when we will have more than 1 neuron in the output layer for recognising letters.

The main risks for the final defense are:

- *Integration.* Modules written separately may disagree on formats. We fix the exchanged data early: SDL surfaces in, letter images to the network, arrays of characters out.
- *Generalization.* A network trained on clean letters may fail on level 2 and 3 images, so the training set must include noisy and rotated samples.
- *Level 3 detection.* Grids without borders, several word columns and extra content are the hardest part, and work on them must start as soon as level 1 works.

Neural Network Section

1. Objectives of the neural network

The final application must recognize each letter of the grid with a neural network. For the first defense, the goal was not yet to read letters but to understand how a multi-layer network works, and to build from scratch, in C and without any library, a network able to learn the XNOR function. For this defense, our objectives were:

Objective	Status
Network structure holding the weights and biases of each layer	Done
Random initialization of the weights and biases	Done
Forward propagation with a sigmoid activation	Done
Backpropagation and gradient descent	Done
Training on XNOR until the loss is below 0.001	Done
Saving and loading the learned parameters	Planned for the final defense
Recognition of letters from images	Planned for the final defense

Gap between plan and result: the XNOR prototype is complete and learns the function. Saving, loading and letter recognition were not expected for this defense. The prototype is written so that they can be added (see 5).

2. Architecture

The neural network is in its own separate folder, since it does not interact yet with the rest of the project. It is made of three files:

- **xnor_network.h** defines the layer sizes (NB_INPUT, NB_HIDDEN, NB_OUTPUT) and the network structure, which holds the weights and biases of the two layers (w1, b1, w2, b2).
- **xnor_network.c** contains the activation function, the initialization, the forward and backward passes, the training loop and main.
- **Makefile**: “all” builds the executable xnor with -Wall -Wextra, and “clean” removes xnor and the results file.

The network has one hidden layer of 3 neurons, 13 parameters in total:

Layer	Neurons	Activation	Parameters
Input	2	none	-
Hidden	3	sigmoid	6 weights, 3 biases
Output	1	sigmoid	3 weights, 1 bias

The program runs as follows. main initializes the network and prints its starting parameters (print_network), then calls update, which trains it. For each sample, forward computes the activations h of the hidden layer and the output o, then backward corrects the parameters from the error on o.

3. Technical choices and justifications

3.1 XNOR as a first problem

XNOR returns 1 when its two inputs are equal and 0 otherwise:

Input A	Input B	Target
0	0	1
0	1	0
1	0	0
1	1	1

The two classes cannot be separated by a straight line, so a single neuron is not enough: a hidden layer is needed. This makes XNOR the smallest problem that really requires backpropagation. In theory 2 hidden neurons are enough; we chose 3 to give the network some margin.

3.2 Sigmoid activation

Both layers use the sigmoid function, sigmoid(x) = 1 / (1 + e^(-x)).

- *Advantage*: the output is between 0 and 1, like our targets, and the function is differentiable. Its derivative is a × (1 - a), where a is the output of the sigmoid itself,

so sigmoid_prime takes the already computed activation and does not need to call exp again.

- *Limitation*: for large positive or negative inputs the sigmoid is almost flat, and the gradient becomes very small. Learning then slows down, which we observe at the end of training.

3.3 Forward propagation

For each hidden neuron j, then for the output, the network computes:

$$z_j = b1_j + w1_{j0} \cdot x0 + w1_{j1} \cdot x1 \qquad h_j = \text{sigmoid}(z_j)$$
$$o = \text{sigmoid}(b2 + w2_0 \cdot h_0 + w2_1 \cdot h_1 + w2_2 \cdot h_2)$$

The hidden activations h are returned to the caller and not kept inside forward, because backpropagation needs them.

3.4 Loss function

The error on one sample is $E = (o - t)^2 / 2$, where t is the target. The factor 1/2 cancels the 2 that appears when E is differentiated. The loss of an epoch is the sum of E over the 4 samples. It is printed every 1000 epochs, and training stops when it falls below 0.001 or after MAX_EPOCHS = 100000 epochs.

3.5 Backpropagation

The error is sent back from the output to the hidden layer with the chain rule. We first compute an error signal for each layer:

$$d2 = (o - t) \cdot o \cdot (1 - o)$$
$$d1_j = d2 \cdot w2_j \cdot h_j \cdot (1 - h_j)$$

Each parameter then moves against its gradient, with a learning rate lr:

$$w2_j = w2_j - lr \cdot d2 \cdot h_j \qquad b2 = b2 - lr \cdot d2$$
$$w1_{ji} = w1_{ji} - lr \cdot d1_j \cdot x_i \qquad b1_j = b1_j - lr \cdot d1_j$$

- *Order of the computation*: the hidden error signals d1 are computed before the output weights are modified, so they use the weights that actually produced the error.

- *Update after each sample*: the parameters are corrected after every sample, not after the 4 samples together (stochastic gradient descent). This is simple to write and works well on a set this small.

- *Learning rate*: fixed at 0.5. With this value the network converges on XNOR.

3.6 Random initialization

Weights and biases are drawn uniformly in [-1, 1] by rand_range, built on rand(). The values must be random: if all weights were equal, all hidden neurons would receive the same

corrections and would stay identical, and the hidden layer would be useless. The generator is not seeded (no srand), so every run starts from the same values. This is convenient for debugging, and a seed can be added when we want different runs.

3.7 Results

Training stops after about 17,700 epochs, when the loss goes below 0.001. The outputs for the four samples are:

Input	Expected output	Output of the network
0 0	1	0.9806
0 1	0	0.0207
1 0	0	0.0225
1 1	1	0.9738

Each output is within 0.03 of its target, so rounding gives exactly the XNOR truth table. The update function also writes the outputs to a file named results.

4. Difficulties and limitations

Limitation	Planned solution
update receives the network by value, so the trained parameters are lost when it returns	Work on the network through a pointer and save the parameters to a file
The output layer is written for one neuron (b2[0], three hard-coded terms in forward), as noted in 3.4	Loop on NB_HIDDEN and NB_OUTPUT everywhere; needed for one output per letter
The random generator is never seeded	Seed it once in main, with an option to fix the seed for reproducible tests
The results file is written at every epoch (about 71,000 lines for one run)	Write it only at the end, or every 1000 epochs like the loss
Sigmoid and a fixed learning rate converge slowly (about 17,700 epochs for 4 samples)	Tune the learning rate and test other activations on the real letter data

5. Next steps

1. Save the learned parameters to a file and load them in the application, so that training is done in a separate program and the application only recognizes.

2. Generalize the code to any layer size, then move to letters: the input is a letter image resized to a fixed size, with one output per letter.

3. Build the training set from letter images, including noisy and rotated samples, to cover the level 2 and 3 images.

4. Write the function that returns the most probable character for a letter image (the output with the highest value).

5. Connect it to the “RUN OCR” button: letter images in, characters out.

6. Bibliography (Neural network)

Michael Nielsen’s book on neural networks and deep learning.

Image Preprocessing and Grid Solver

Image Preprocessing

1.1 Objective

The preprocessing stage turns any input picture (a scan, a photograph, a colour image) into a clean, standardised image on which the next stages can work reliably: locating the grid and the word list, cutting out each letter, and recognising it with the neural network. These stages need an image with one intensity channel, good contrast, little noise, a straight grid and a clear separation between ink and background. The preprocessing stage is responsible for producing such an image.

1.2 Function plan

Function	Role	Status
image_load	Loads a file (PNG, JPEG, BMP) and converts it to grayscale	Done
image_save	Saves an image as a PNG file	Done
image_free	Releases an image and its pixel buffer	Done
automatic_threshold_calculator	Computes a binarisation threshold with Otsu's method	Done
image_binarize	Converts a grayscale image to black and white	Done
image_crop	Extracts a rectangular area (used to isolate letters)	Done
image_rotate	Manual rotation around the centre (described in another section)	Done

Function	Role	Status
image_denoise	Removes grain and small stains	Planned
image_enhance_contrast	Improves the contrast of faded images	Planned
image_detect_skew	Estimates the rotation angle of the grid	Planned

The planned pipeline is: loading, denoising, contrast enhancement, deskew, threshold computation, binarization. The threshold is not computed inside image_binarize: the caller must call automatic_threshold_calculator first and pass the result, which keeps both steps independent and testable.

1.3 Data structure

All functions share one structure:

```
struct Image {
    int width;
    int height;
    unsigned char *pixels;
};
```

pixels is a flat, row-major buffer of width * height bytes. Each byte is a grayscale value, where 0 is black and 255 is white. Pixel (x, y) is at index y * width + x. We chose this format for three reasons: SDL2 is mandatory but we wanted the other modules to be independent from it, color is not needed for character recognition, and a flat buffer is compact and fast to traverse. Every transformation returns a **new** image and never modifies its input, so steps can be chained and compared. The caller owns each returned image and must free it.

1.4 Functions in detail

image_load. The file is decoded with IMG_Load from SDL2_image, which supports several formats. The resulting surface is converted to a fixed RGB24 format, so that the pixel layout is the same whatever the source file. Each pixel is then converted to gray with the luminosity formula 0.299 R + 0.587 G + 0.114 B, rounded to the nearest integer. The row offset uses the surface pitch (the real number of bytes per row, which may include padding) and not width * 3. Using the width instead gave a distorted image. The SDL surfaces are freed before returning, so no SDL type leaves this function. It returns NULL on failure.

image_save. SDL cannot write a raw buffer, so the function wraps pixels in an 8-bit indexed surface, without copying it. It attaches a 256-entry palette where entry i is the colour (i, i, i), then writes the file with IMG_SavePNG. PNG was chosen because it is lossless, unlike JPEG, which would add artefacts to an image that is analyzed afterwards. It returns 0 on success and -1 on error.

image_free. It frees the pixel buffer first and the structure second (the reverse order would lose the pointer). Passing NULL is allowed and does nothing.

automatic_threshold_calculator. It implements Otsu's method. A 256-bin histogram is built, then every candidate threshold is tested. For each one, the pixels are split into a dark group (values below the threshold) and a light group, and the function computes the

between-class variance, w0 * w1 * (mean0 - mean1)^2. The threshold that maximises it is returned. Cumulative counts and sums are updated at each step, so the algorithm is linear in the number of pixels. Sums use long to avoid overflow on large images. It returns -1 if the image is invalid. The method gives one global threshold, so it works well on evenly lit images but not on images with shadows.

image_binarize. Each pixel below the threshold becomes 0 (black); all others become 255 (white). The comparison is strict, and it is consistent with the dark group used by Otsu's method.

image_crop. It copies the rectangle (x, y, w, h) into a new image. Arguments are checked first: a null image, non-positive dimensions or a rectangle outside the image give NULL. The destination uses its own width (w) as the row stride, and the source uses width. Mixing them up was a bug we fixed, because it writes outside the buffer. This function will be called for every letter.

1.5 Planned functions

- **image_denoise**: a 3×3 median filter. Each pixel takes the median of its neighbourhood, which removes isolated dark or light points while keeping the edges of letters sharp. A blur would have spread the noise and softened the letters. Border pixels are copied unchanged. It runs before the threshold computation, because noise distorts the histogram.

- **image_enhance_contrast**: histogram stretching. The darkest and brightest values (ignoring a small percentage of extremes) are mapped to 0 and 255, and the others are rescaled linearly. This helps Otsu's method on faded images (level 2). Histogram equalization is kept as an alternative.

- **image_detect_skew**: the angle is estimated with a projection profile. The image is rotated by candidate angles, and the angle that maximizes the variance of the row sums is kept, because text lines and grid lines are sharpest when horizontal. The result is applied with image_rotate. A Hough transform on the grid lines is the alternative.

We considered using a neural network that locates the grid. We did not choose it for deskew, because locating the grid gives a bounding box and not an angle. A network would need to be trained to predict the angle, which requires a large labelled set of rotated images and would add risk to the project. The classical methods need no training data, are deterministic and are easy to explain.

2. Grid Solver

2.1 Objective

Once the letters have been recognized, the grid is rebuilt as an array of characters and each word of the list has to be found in it. The solver returns the position of the first and the last

letter of every word, which is used both by the command-line solver and by the display of the circled words.

2.2 Function plan

Function	Role	Status
initialize_grid	Builds a grid from a character string	Done
free_grid	Releases a grid	Done
find_word	Searches one word and returns its position	Done
solve_grid	Searches every word of a list	Done

2.3 Data structures

```
struct Grid {
    int width;
    int height;
    char *table;
};

struct Match {
    int x1, y1;
    int x2, y2;
};
```

Grid stores the letters in uppercase, row by row, in table (width * height characters, without a terminating \0). Match holds the coordinates of the first letter (x1, y1) and of the last letter (x2, y2), where x is the column, y is the row and (0, 0) is the top-left cell, as required by the subject.

2.4 Functions in detail

initialize_grid. It takes the source string, its length and the dimensions w and h. It checks that the string contains at least w * h characters, allocates the structure and its table, and copies the first w * h characters converted to uppercase. It returns NULL on error.

free_grid. It frees the table and then the structure.

find_word. It returns 1 if the word is found (and fills the Match), 0 if it is not found and -1 on error. The algorithm goes through every cell of the grid. When the cell contains the first letter of the word, it examines the 8 neighbouring cells. If a neighbour contains the second letter, the direction is fixed and the function follows it, comparing one letter at a time, until a letter differs or the end of the word is reached. If the whole word matches, the match is returned. Otherwise the remaining directions around that first letter are tested, and then the scan continues with the next cell. The worst-case cost is O(w * h * 8 * L), where L is the length of the word, but in practice most cells are rejected after a single comparison. A word running past the border of the grid stops the loop and is rejected.

solve_grid. It runs find_word once for each word of the list and returns the array of matches, so that the interface can draw the result and the solver can print the coordinates.

UI Section

1. Objectives of the interface

The subject asks for a graphical interface that lets the user load an image, view it, correct some defects, then display and save the solved grid. For the first defense, our objectives were:

Objective	Status
SDL2 window with event loop and rendering	Done
Loading an image from a path (typed or pasted)	Done
Image preview, scaled without distortion	Done
Manual rotation by an angle entered by the user	Done
Saving the displayed image as PNG	Done
Preprocessing checkboxes (auto deskew, noise reduction, contrast)	Interface ready, processing not connected
“RUN OCR” and “SOLVE GRID” buttons	Interface ready, modules not connected
Display of the recognized grid and the solved grid	Planned for the final defense

Gap between plan and result: drag and drop of a file onto the window was planned at first. It was dropped (see 3.4) and replaced by typing or pasting the path.

2. Architecture

The interface code is split into two modules:

- **`ui.c`** handles only the interface: state (`UiState`), events, and drawing the panels.
- **`bitmap.c`** handles image processing: `image_load`, `image_rotate`, `image_save`, `image_clean_path`.

`ui.c` contains no neural-network code. The “RUN OCR” button is the single entry point to the recognition pipeline. The interface passes it the pixels and displays the results, so each team member can work on their own module without touching the others.

Two copies of the image are kept in `UiState`:

- `source`: the original image, never modified;
- `current`: the image after correction, which is displayed, saved, and meant for the pipeline.

3. Technical choices and justifications

3.1 SDL2 without SDL_ttf: a built-in pixel font

Text is drawn with a small built-in font: each letter is an array of 7 rows of 5 bits, and each set bit becomes a `scale × scale` square of pixels.

- *Advantage*: no extra dependency and no font file to ship. The repository must contain only necessary files, and the program must run on the school machines.
- *Limitation*: only letters, digits and a few symbols (`- _ . : /`) exist, and the rendering is basic.

3.2 Clickable areas defined once

Each clickable area is a single `SDL_Rect` constant, used both for drawing and for click detection. We first wrote the coordinates twice, once in the drawing code and once in the click code. They drifted apart, and the checkboxes only reacted on their upper half. A single definition removes this whole class of bugs.

3.3 Rotation by inverse transformation

Rotation is written by hand in `image_rotate`, without an external library. For each pixel of the new image, we compute where it comes from in the original image:

```
x_src =  cos(θ)·dx + sin(θ)·dy + center_x
y_src = -sin(θ)·dx + cos(θ)·dy + center_y
```

where `dx` and `dy` are the coordinates relative to the center of the new image.

- *Why inverse?* With the forward method (sending each original pixel to its new position), rounding leaves holes in the image. With the inverse method, every destination pixel receives exactly one value.
- *Canvas size*: the new width is `|w·cosθ| + |h·sinθ|` and the new height is `|w·sinθ| + |h·cosθ|`, so the corners are not cut off. Empty areas are filled with white, a neutral color for a document.

- *Always rotating from* `source`: applying several successive rotations would degrade the image each time. Starting from the original guarantees that an angle of 0 gives back the exact image.
- *Interpolation*: we use nearest neighbor (fast, slightly jagged). Bilinear interpolation can be added if recognition suffers.

3.4 Loading: typing, pasting, and dropping drag and drop

The path can be typed in the input field, or pasted with Ctrl+V. A pasted path is cleaned by `image_clean_path`: whitespace, newlines and quotes are removed, the `file://` prefix is stripped, and `%XX` sequences are decoded (for example `%20` for a space).

Drag and drop (`SDL_DROPFILE`) did not work in our test environment. This behavior depends on the window manager and the system (virtual machines, Wayland), which do not always forward the event. Since pasting a path works everywhere, we removed this feature and disabled those events (`SDL_EventState`) rather than ship an unreliable one.

3.5 Image formats and conversion to RGBA32

Loading uses `SDL2_image` (`IMG_Load`), which reads PNG and JPEG. Each image is then converted to a single pixel format (RGBA32, 32 bits). This lets the rotation copy pixels as 32-bit integers, without handling each file format separately. Future preprocessing (grayscale, black and white) will also start from this format.

3.6 Error handling without quitting the program

The functions in `bitmap.c` return `NULL` or an error code instead of calling `errx`. A typo in the path therefore does not close the window: the message “COULD NOT LOAD IMAGE - CHECK THE PATH” appears in the status bar. The old functions that exit the program are kept for command-line tests, but the interface never calls them.

3.7 Saving

The “SAVE IMAGE” button writes the currently displayed image (`current`) to `output.png` with `IMG_SavePNG`. PNG is lossless, which suits an image later used for recognition. For the final defense, this button will save the solved grid with the words circled.

4. Difficulties encountered

Difficulty	Solution
Checkboxes not clickable on their lower half	Single rectangles shared by drawing and clicking
The `_` character in `level_1_image_1.png` was displayed as a space	Added `_` to the font

The `/` character was drawn mirrored	Fixed the bit shift
Drag and drop not working	Replaced by typing and pasting the path
Pixels were freed after creating the texture, so the pipeline could not use them	Kept `source` and `current` in memory
`ui.c` was becoming too long	Moved image processing into `bitmap.c`
Link errors (`-lSDL2_image`, `-lm`, `bitmap.c` missing from the sources)	Fixed the Makefile

5. Next steps

6. Connect grayscale, noise reduction, contrast enhancement and automatic angle detection (the checkboxes are already in the interface).
7. Display the recognized letters in the grid and the word list, once the recognition module is available.
8. Draw the solved grid with each word circled, using the first and last letter positions returned by the solver.
9. Add an editable output file name.

6. Bibliography (UI)

- SDL2 documentation: https://wiki.libsdl.org/SDL2/FrontPage
- SDL2_image documentation: https://wiki.libsdl.org/SDL2_image/FrontPage
- Image rotation by inverse transformation: image processing course (complete with the source you actually used)